

数据结构与算法 K03 课堂练习

1900011604 张植竣 指导老师：陈斌

北京大学物理学院天文学系

2020 年 3 月 10 日

1 代码

```
def calculate(s): #s为需要计算的后缀表达式字符串

class Stack: #定义数据类型：栈

    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[len(self.items) - 1]

    def size(self):
        return len(self.items)

    def __repr__(self): #将栈转化为字符串由print()函数调用输出
        if len(self.items) != 0:
            s='[ '
            for i in range(len(self.items)-1):
                a=self.items[i]
                s+=' %s | ' %a
            s+=' %s'%self.items[len(self.items) - 1]
        else:
            s='[ '
        return s
```

```

def calc(num0, op, num1): #计算表达式
    num0 = int(num0)
    num1 = int(num1)
    if op == '+':
        return num0 + num1
    elif op == '-':
        return num0 - num1
    elif op == '*':
        return num0 * num1
    elif op == '/':
        return num0 / num1

slist = s.split() #将字符串转化为列表，方便操作
slist.reverse() #因为栈具有LIFO的特性，需要将slist倒序
x = Stack()
for i in range(len(slist)):
    x.push(slist[i])

y = Stack()
y.push(x.pop())
while x.size() > 0:
    while y.peek() not in "+-*/":
        y.push(x.pop())
    print(y) #将操作数和操作符从储存栈x压入计算栈y

    op = y.pop()
    num1 = y.pop()
    num0 = y.pop()
    y.push(str(calc(num0, op, num1))) #进行表达式的计算，并将结果压入栈顶
    print(y)
print('')

calculate('2 3 * 4 +')
calculate('1 2 + 3 + 4 + 5 +')
calculate('1 2 3 4 5 * + * +')

```

2 计算"2 3 * 4 +"的输出结果

```

[ 2 | 3
[ 2 | 3 | *
[ 6
[ 6 | 4
[ 6 | 4 | +
[ 10

```

3 计算"1 2 + 3 + 4 + 5 +" 的输出结果

```
[ 1 | 2
[ 1 | 2 | +
[ 3
[ 3 | 3
[ 3 | 3 | +
[ 6
[ 6 | 4
[ 6 | 4 | +
[ 10
[ 10 | 5
[ 10 | 5 | +
[ 15
```

4 计算"1 2 3 4 5 * + * +" 的输出结果

```
[ 1 | 2
[ 1 | 2 | 3
[ 1 | 2 | 3 | 4
[ 1 | 2 | 3 | 4 | 5
[ 1 | 2 | 3 | 4 | 5 | *
[ 1 | 2 | 3 | 20
[ 1 | 2 | 3 | 20 | +
[ 1 | 2 | 23
[ 1 | 2 | 23 | *
[ 1 | 46
[ 1 | 46 | +
[ 47
```
